ASSINGMENT 1: Spotify Data Base
ATHANASIOS GOURDOMICHALIS

1. Objective

In this assignment, we will design and implement the relational tables for a music database using SQLite. The database stores information regarding artists, albums, tracks, music genres, and audio features (such as popularity, tempo, etc). To define the database schema, we must map the attributes based on the input data types present in the provided CSV files. Data population must be performed using the .import command of the SQLite command-line tool.

2. Data Description

The raw data contains information on 10,000 Spotify albums, along with associated artists, tracks, audio features, and music genres. The schema consists of the following tables:

- **albums**: Contains album entities with attributes: id, name, album_type, release_date, and popularity.
- **artists**: Contains artist entities with attributes: id, name, popularity, and followers.
- **audio_features**: Contains acoustic metrics of tracks, such as acousticness, danceability, energy, loudness, tempo, and others.
- **genres**: Contains music genres, stored by genre name.
- **tracks**: Contains track entities with attributes: id, name, duration, track_number, popularity, and preview.
- **r_albums_artists**: A table linking albums to their respective creators, containing foreign keys album_id and artist_id.
- **r_albums_tracks**: A table linking albums to their tracks, containing foreign keys album_id and track_id.
- **r_artist_genre**: A table linking artists to their music genres, containing foreign keys genre_name and artist_id.
- **r_track_artist**: A table linking tracks to the performing artists, containing foreign keys track_id and artist_id.

3. Implementation Steps

We will implement a local SQLite database and import the data from the CSV files to simulate a Spotify-like application backend. The database must be stored in a file named exercise2.db. Each table in the database will be populated from its corresponding CSV file.

Steps we follow:

1. **Database File Creation**: Initialize the SQLite database file using the command: ***sqlite3 exercise2.db***
2. **Schema Definition:**
   - Define the relational schema using SQL CREATE TABLE DDL statements.
   - **Warning**: Since SQLite has limited native support for the ALTER TABLE statement, all integrity constraints must be declared directly within the initial CREATE TABLE block of each table. We will not use the ALTER TABLE command to append primary or foreign keys.
   - We ensure that every SQL statement is terminated with a semicolon (;).
   - Constraints:

i. We define a **Primary Key** for the following tables: **albums, artists, audio_features, genres, and tracks.**

ii. We define **Foreign Keys** for the relationship tables: **r_albums_artists, r_albums_tracks, r_artist_genre, and r_track_artist.**
For these junction tables, the primary key must be a **composite primary key** consisting of the combination of their two foreign keys.

iii. In the audio_features table, we define a **Foreign Key** referencing the tracks table. This specific foreign key must also serve as the table's **Primary Key** (establishing a 1:1 relationship).

iv. We save all our DDL statements in a file named create_table.sql (we can use the .schema command within the SQLite command-line interface to inspect and extract our definitions).

3. **Data Importing:**

    a. We create a script file named **imports.sql** containing all the **.import** meta-commands.
    b. We set the shell mode to CSV at the beginning of the script with the SQL command:
    .mode csv
    c. We include a **.import** statement mapping each CSV file to its designated table.
    d. **Tip:** If the CSV source files contain headers, use the following SQL command to skip the first row:
    .import --skip 1 file.csv *table_name*

4. Useful Links:

- **Official Documentation:** SQLite Documentation
- **Create Table Statment: Syntax and Constraints:** CREATE TABLE
- **Command-line Interface Guide for the .import command:** Command Line Shell For SQLite

5. Project Files

- **create_table.sql**: Containing all the DDL (Data Defining Language) CREATE TABLE commands.
- **imports.sql**: Containing all the SQL CLI (command-line interface) commands to import the dataset into the tables.